SESSION 3: DATA VISUALIZATION

DR. SOFIA GIL-CLAVEL

- Recap
- Ggplot2 & the Grammar of Graphs

1. RECAP

Data Types

Primitive

Composite

Integers

Float/ Doubl

Character & String

Boolean

Vector

Data Frame

```
> n = c(2, 3, 5)
> s = c("aa", "bb", "cc")
> b = c(TRUE, FALSE, TRUE)
> df = data.frame(n, s, b)
```

PRIMITIVE DATA AND ITS OPERATORS

Operators					
Arithmetic		**Comparative**		**Logic**	
+	addition	<	less than	! x	logic NO
-	subtraction	>	more than	x & y	element-wise AND
*	multiplication	<=	less or equal than	x && y	single comparison AND
/	division	>=	more or equal than	x | y	element-wise OR
^	power	==	equal than	x || y	single comparison OR
%/%	integer division	!=	different than	xor(x,y)	exclusive OR

❖ DATAFRAME FUNCTIONS

The dataframe name

The columns:
names(mtcars)
or
colnames(mtcars)

```
> mtcars
                   mpg cyl disp  hp drat   wt ...
Mazda RX4         21.0   6  160 110 3.90 2.62 ...
Mazda RX4 Wag     21.0   6  160 110 3.90 2.88 ...
Datsun 710        22.8   4  108  93 3.85 2.32 ...
                  ...........
```

The row names:
row.names(mtcars)

A variable:
mtcars$disp

A value or element:
mtcars$wt[3]

SOME USEFUL FUNCTIONS

`sum(x)`
`prod(x)`
`max(x)`
`min(x)`
`which.max(x)`
`which.min(x)`
`range(x)`
`length(x)`
`mean(x)`
`median(x)`
`var(x)` o `cov(x)`
`cor(x)`
`var(x, y)` o `cov(x, y)`
`cor(x, y)`

`round(x, n)`
`rev(x)`
`sort(x)`
`rank(x)`
`log(x, base)`
`scale(x)`
`pmin(x,y,...)`
`pmax(x,y,...)`
`cumsum(x)`
`cumprod(x)`
`cummin(x)`
`cummax(x)`

❖ TIDYVERSE: HTTPS://WWW.TIDYVERSE.ORG/

R packages for data science

The tidyverse is an opinionated **collection of R packages** designed for data science. All packages share an underlying design philosophy, grammar, and data structures.

Install the complete tidyverse with:

```
install.packages("tidyverse")
```

DATA PIPELINES

1.

```
> iris
# A tibble: 150 × 5
   Sepal.Length Sepal.Width Petal.Length Petal.Width Species
          <dbl>       <dbl>        <dbl>       <dbl> <fct>
 1          5.1         3.5          1.4         0.2 setosa
 2          4.9         3            1.4         0.2 setosa
 3          4.7         3.2          1.3         0.2 setosa
 4          4.6         3.1          1.5         0.2 setosa
 5          5           3.6          1.4         0.2 setosa
 6          5.4         3.9          1.7         0.4 setosa
 7          4.6         3.4          1.4         0.3 setosa
 8          5           3.4          1.5         0.2 setosa
 9          4.4         2.9          1.4         0.2 setosa
10          4.9         3.1          1.5         0.1 setosa
# ℹ 140 more rows
# ℹ Use `print(n = ...)` to see more rows
```

2.

Basic piping

- `x %>% f` is equivalent to `f(x)`
- `x %>% f(y)` is equivalent to `f(x, y)`
- `x %>% f %>% g %>% h` is equivalent to `h(g(f(x)))`

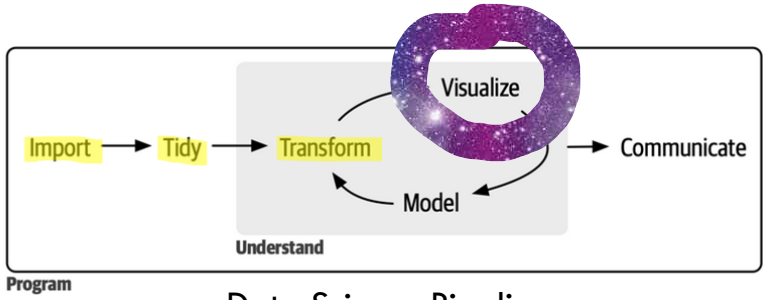
3.

dplyr is a grammar of data manipulation, providing a consistent set of verbs that help you solve the most common data manipulation challenges:

- `mutate()` adds new variables that are functions of existing variables
- `select()` picks variables based on their names.
- `filter()` picks cases based on their values.
- `summarise()` reduces multiple values down to a single summary.
- `arrange()` changes the ordering of the rows.

2. GGPLOT2 & THE GRAMMAR OF GRAPHS

❖ GGPLOT: HTTPS://WWW.TIDYVERSE.ORG/



Data Science Pipeline

ggplot2 is a system for declaratively creating graphics, based on The Grammar of Graphics. You provide the data, tell ggplot2 how to map variables to aesthetics, what graphical primitives to use, and it takes care of the details.

Source: https://ggplot2.tidyverse.org/

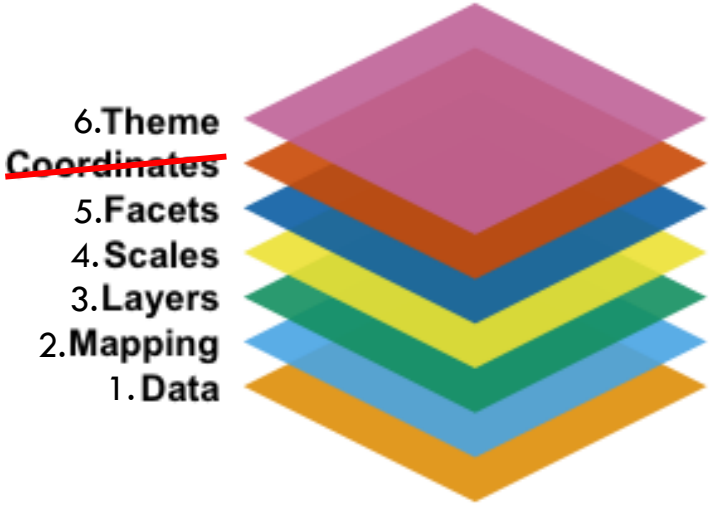
❖ GGPLOT2

ggplot2 is a package for creating graphs, which follows the grammar of graphs.

What is the grammar of graphs?

The grammar of graphs tells us that "... Every statistical graph is a mapping of data to a set of geometric objects (points, lines, bars) that contain aesthetic attributes (color, shape, size). The graph can even have statistical transformations of the data, and these are drawn on a specific Cartesian plane."

THE COMPONENTS OF A GRAPH



1) **Data** is the foundation of every graphic. The system works best if the **data is provided in a tidy format**, which briefly means a rectangular data frame structure where rows are observations and columns are variables.

2) The **mapping** of a plot is a set of instructions on how parts of the data are mapped onto aesthetic attributes of geometric objects. It is the 'dictionary' to translate tidy data to the graphics system.

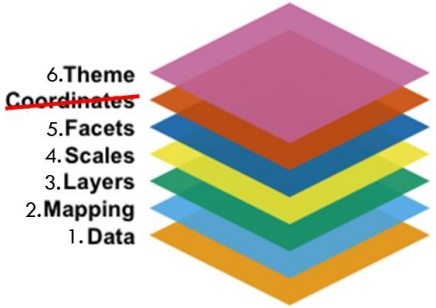
3) **Layers** made of geometric objects (which we'll call **geoms**) and statistical transformations (which we'll call **stats**). Geoms represent what you see on the graph: points, lines, polygons, etc. Stats are a summary of the data being observed.

4) **Scales** maps the values of the data to values in an aesthetic space, such as color, size, or shape. Scales draws the legends or axes, which provides reverse mapping that makes it possible to read the original data from the graph.

5) The **facet** specification describes how to divide data into subsets and how to display it in subgraphs within the same graph. This is also called a conditioner.

6) A **theme** controls the points that are displayed, such as font size and color.

A quick guide is here: https://github.com/rstudio/cheatsheets/blob/main/data-visualization.pdf



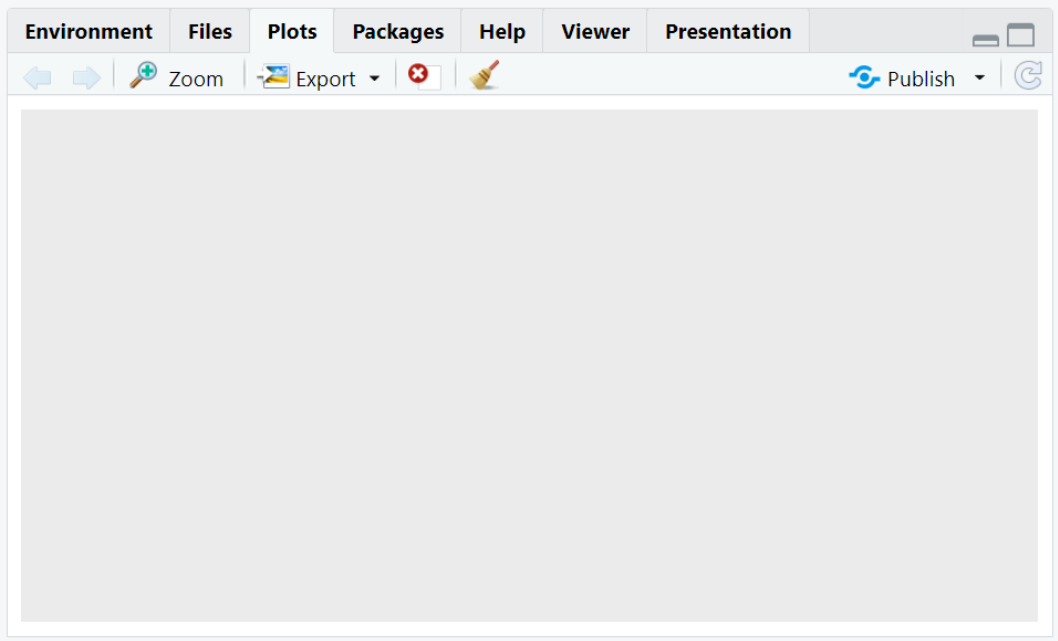
1) DATA

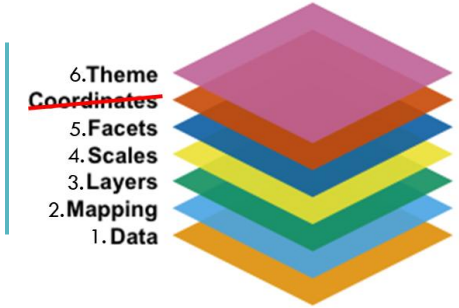
This is just the Data Frame that you want to visualize.

```
library(tidyverse)
IRIS<-tibble(iris)
```

1.Data

```
IRIS%>%
  ggplot()
```



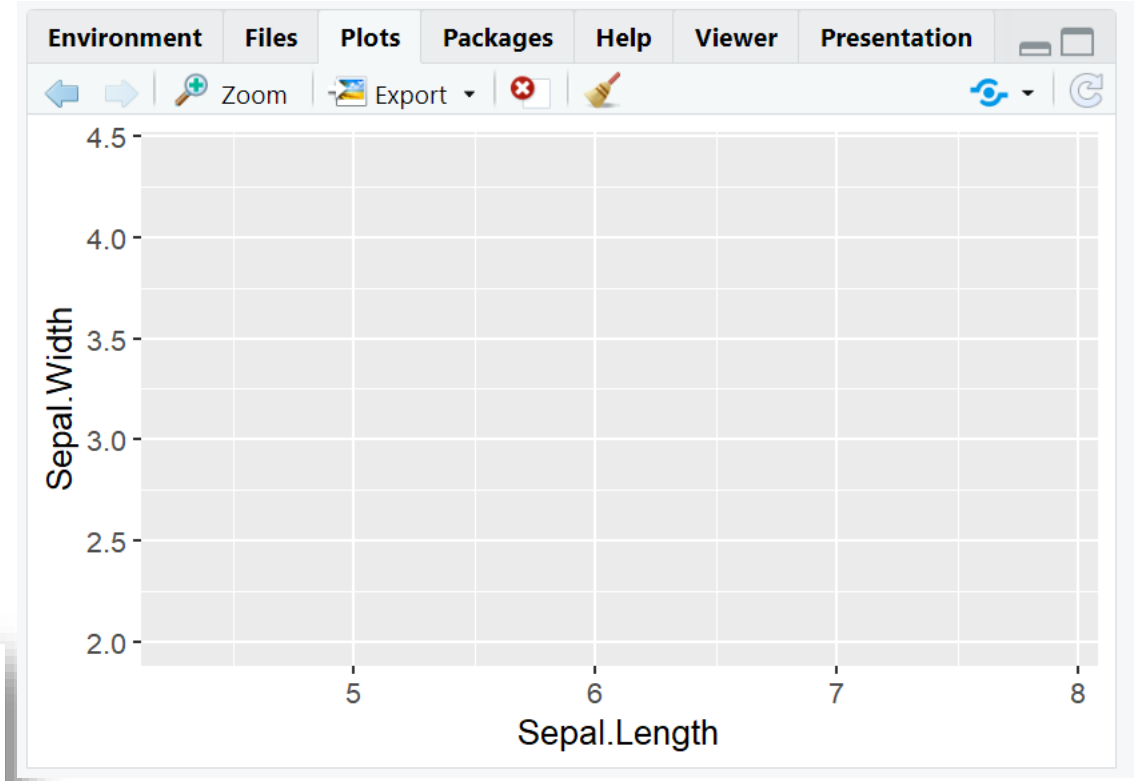


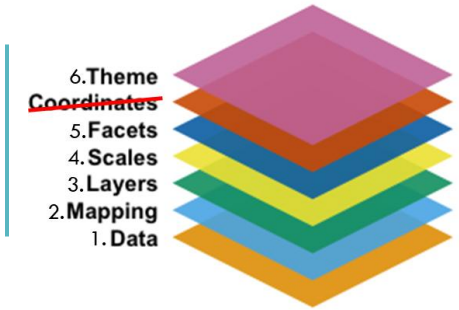
2) MAPPING

This refers to the variables you want to map: X and Y.
This is done by nesting the function aes() inside the function ggplot().
Inside the function aes(), you must add the names of the columns (X and/or Y) from your data that you want to plot.

2. Mapping:
Variables in the Data

```
IRIS%>%
  ggplot(aes(x=Sepal.Length,y=Sepal.Width))
```





3) LAYER: GEOMS

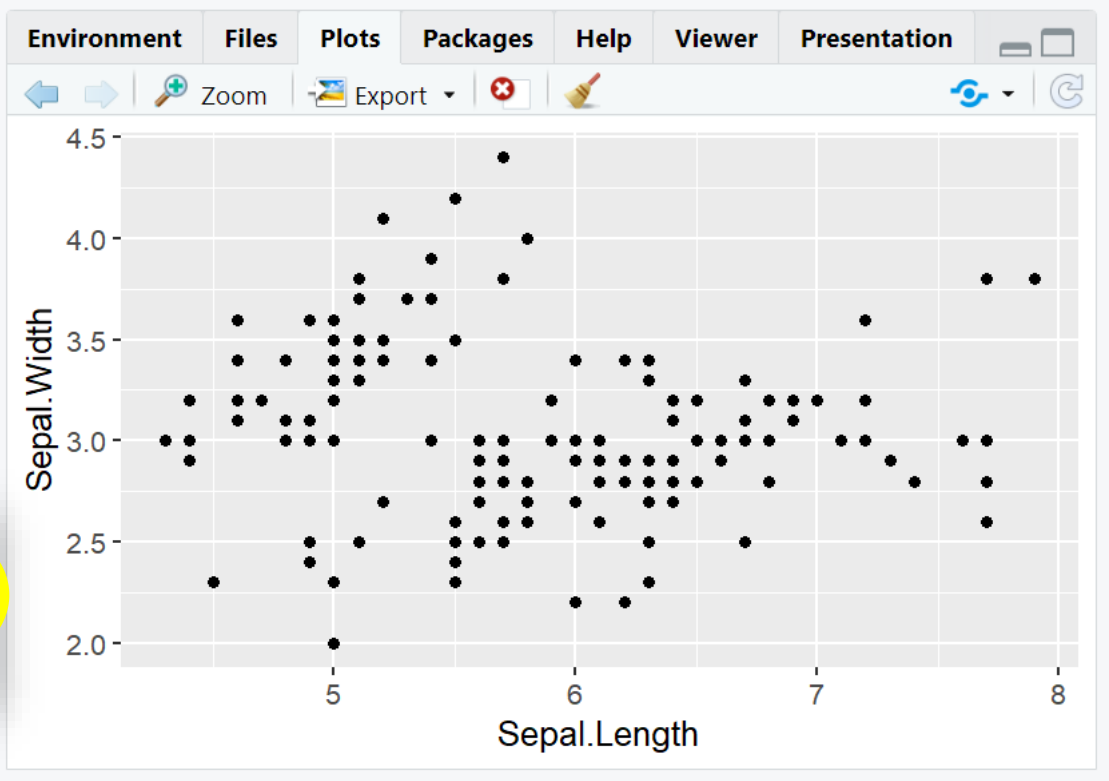
This refers to how the visualization between the variables (X and Y) will be done. You can visualize it as dots, or as bars, or as whatever you think fits.

In Rstudio, you can see all the geometries by typing "geom_", followed by pressing tap on your keyboard.

```
IRIS%>%
  ggplot(aes(x=Sepal.Length,y=Sepal.Width)) +
  geom_point()
```

3. Scale:
Geometric Object

Add and transform the graph



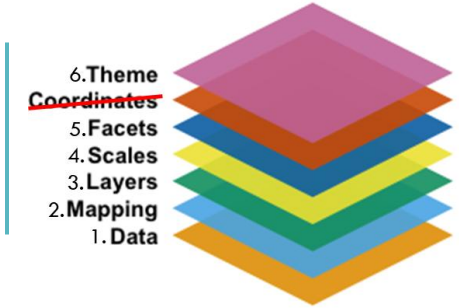
EXERCISE 2.1

- Open your favorite browser and look for the ggplot2 Cheat Sheet.

Hint: Use the key words Possit, ggplot2, cheat sheet.

- Produce the following three graphs:

1. The histogram of the variable Petal.Width.
2. The barplot of the variable Species.
3. The boxplot of the variable Petal.Width by Species.

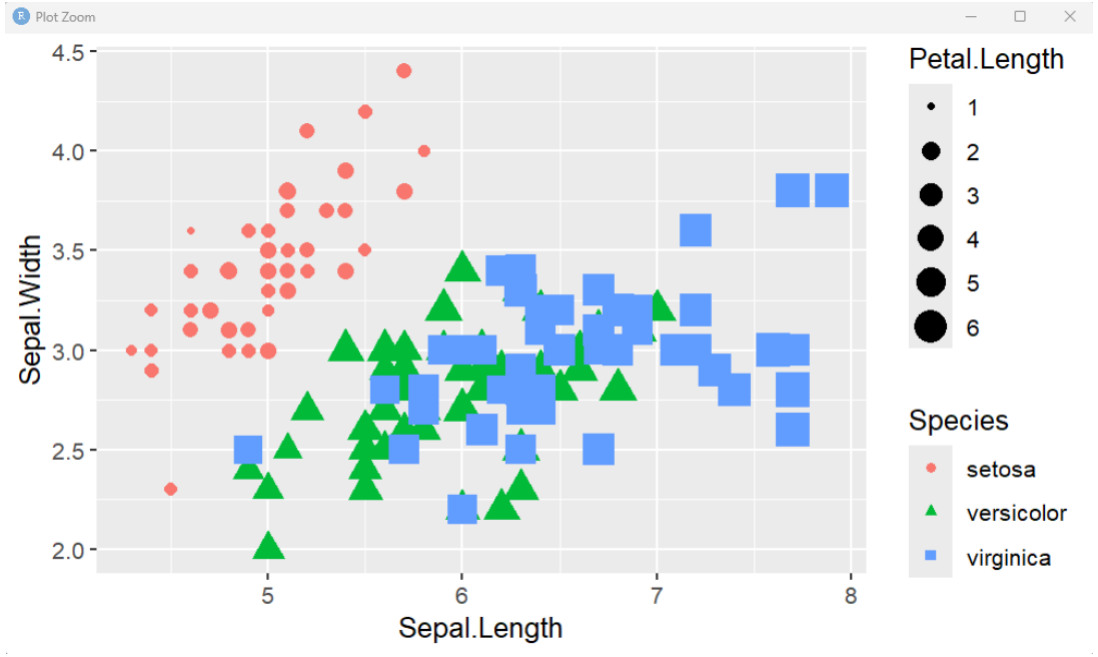


4) SCALES: COLOR, SIZE, SHAPE, AND OTHER AESTHETIC ATTRIBUTES

Let's say you chose to visualize the relation between your variables as dots. The scales refer to aesthetic changes to these dots. For example, you can choose to visualize the dots relative to a categorical variables (Z). So, instead of only black dots, you would have other shapes and colors depending on Z. However, their location would remain based on the value of X, Y.

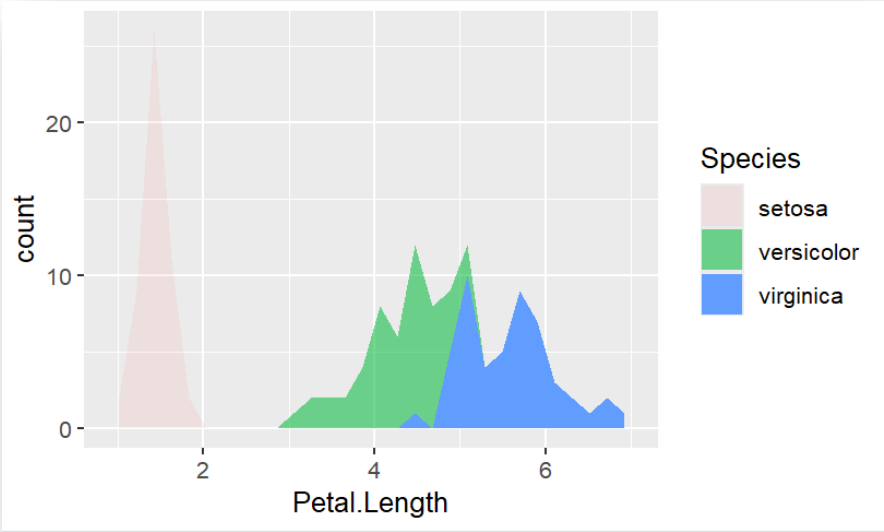
```
IRIS%>%
  ggplot(aes(x=Sepal.Length,y=Sepal.Width,
             color=Species,
             shape=Species,
             size=Petal.Length  ))+
  geom_point()
```

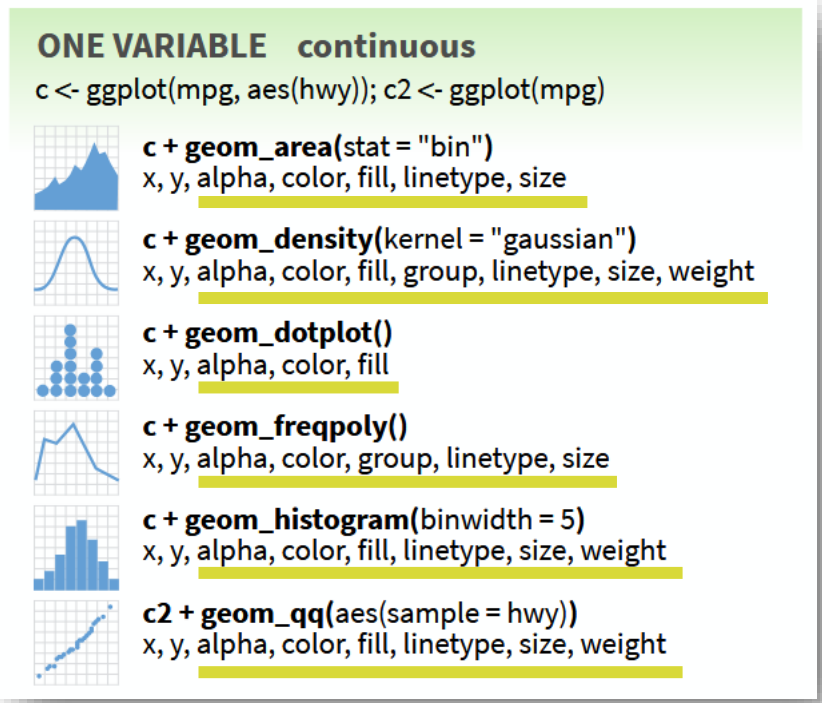
4. Scales



EXERCISE 2.2

- In the ggplot2 Cheat Sheet, you can check the scales that can be changed depending on the plot.
- Using the information in the box "ONE VARIABLE continues", try to produce the following graph:





4) SCALES: SUBGROUPS POSITIONS

- `position_stack()`: stack overlapping bars (or areas) on top of each other.
- `position_fill()`: stack overlapping bars, scaling so the top is always at 1.
- `position_dodge()`: place overlapping bars (or boxplots) side-by-side.

```
geom_bar()
geom_bar(position = "fill")
geom_bar(position = "dodge")
```

The same happens with geom_col().

EXERCISE 2.3

Some data handling with dplyr

1. Group your data by Species.

   Hint: Use the function group_by().

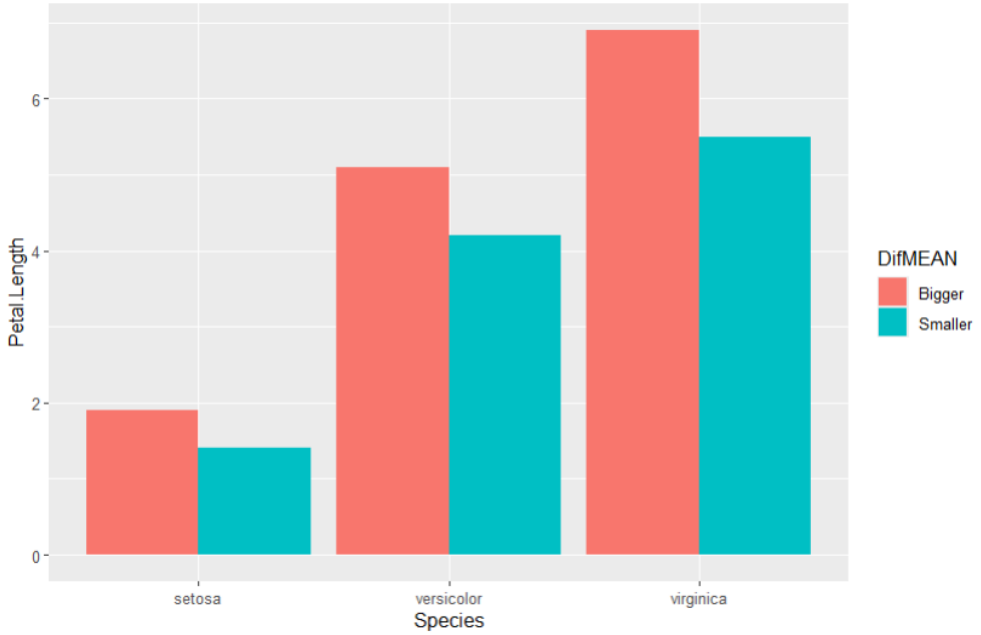
2. Create a new variable (DifMEAN) to store whether the values in Petal.Length are smaller, equal, or bigger than the average of the group.

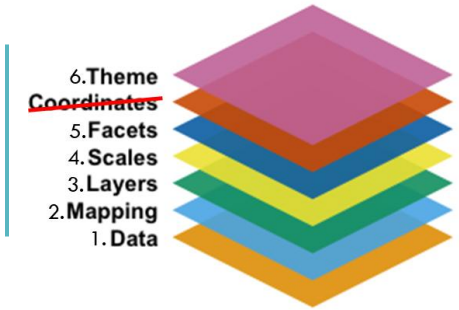
   Hint: Use the functions mutate(), ifelse(), and mean().

Visualizing the groups with ggplot2

Try to produce the following graph:

Hint: Use the subgroups positions.

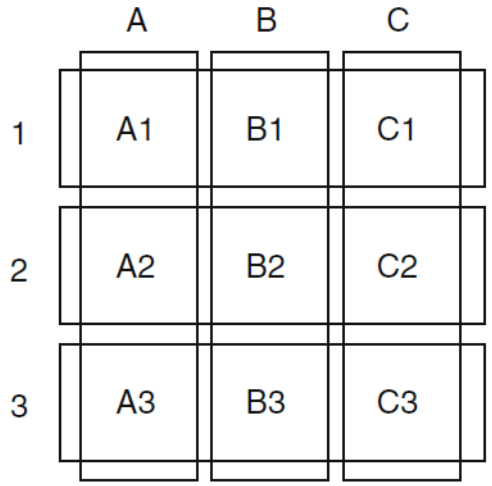




5) FACETS

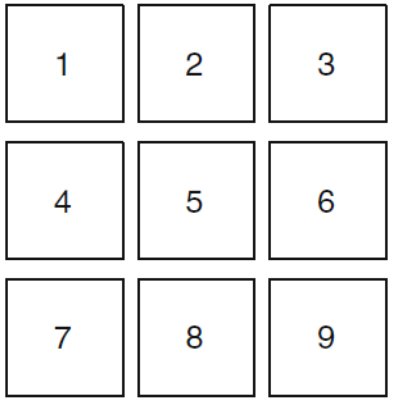
```
ONE%>%
  ggplot(aes(Species,Petal.Length))+
  geom_col(position = "dodge")+
  facet_grid(DifMEAN~.)
```

Faceting refers to creating the same graph but splitting it by specific categorical variables. The functions are facet_grid(), and facet_wrap(). As parameter, you need to pass the variables that will be used to split the plots: **X1+X2~Y1+Y2**



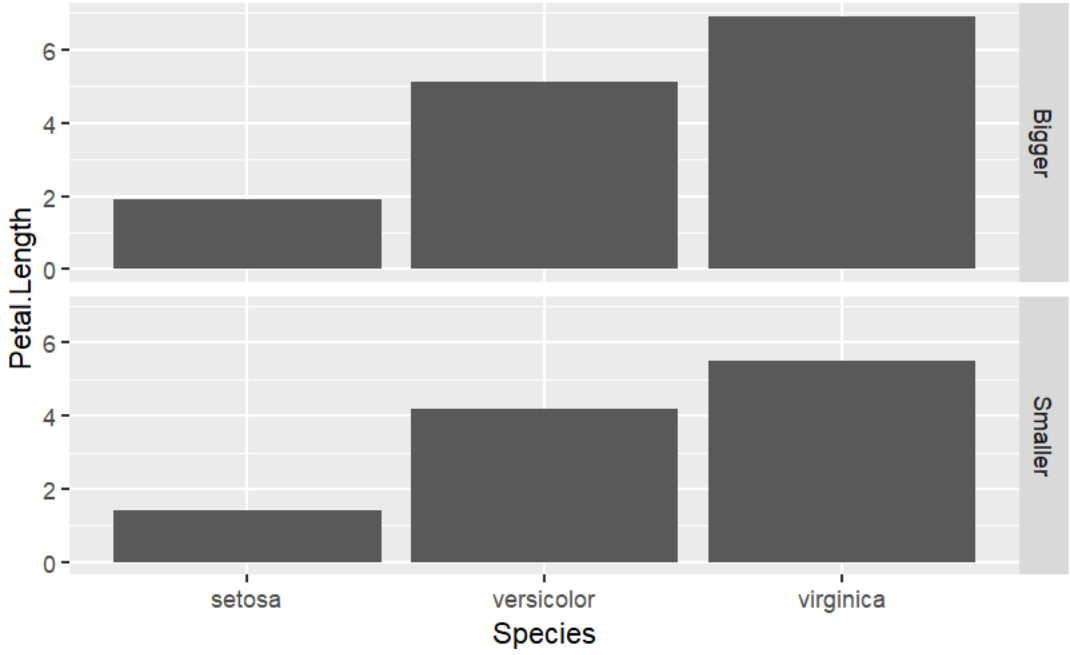
facet_grid

They share axes



facet_wrap

Each chart has its axes



EXERCISE 2.4

Some data handling with dplyr

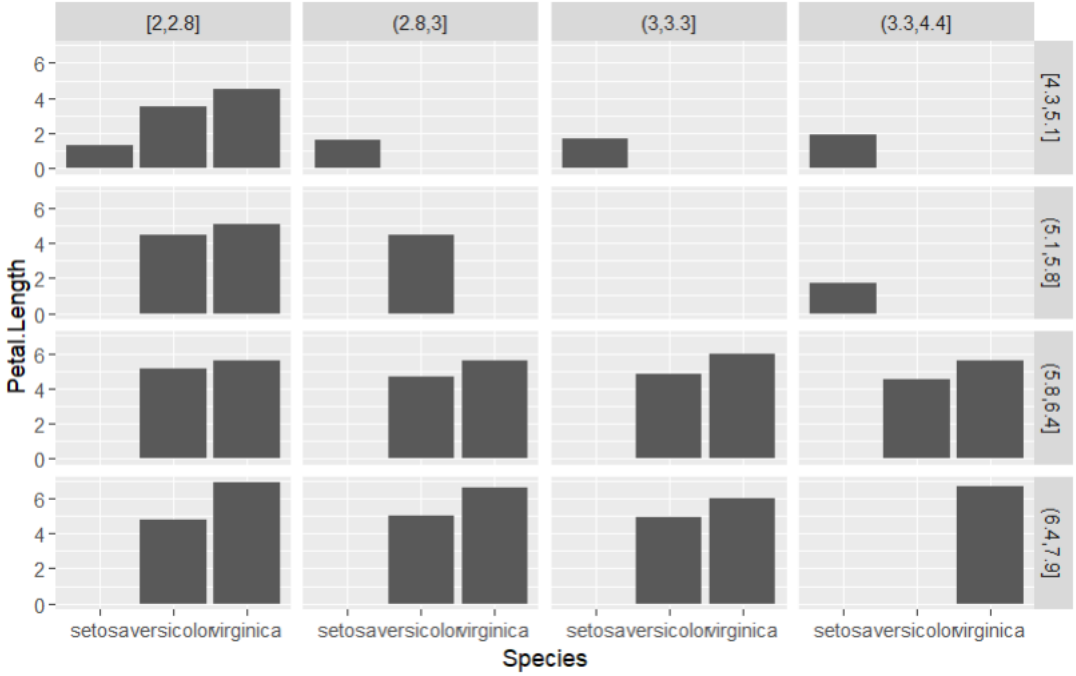
1. Using the function cut() and quantile() create a new variable indicating to what quantile the value in Sepal.Length belongs to.
2. Using the function cut() and quantile() create a new variable indicating to what quantile the value in Sepal.Width belongs to.

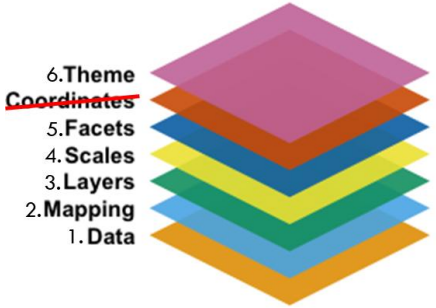
Hint: In both cases, use the parameter "include.lowest = TRUE" for the function cut().

3. Save the results in the variable EX_3.4.

Visualizing the groups with ggplot2

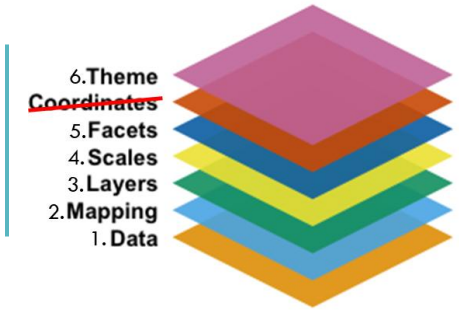
Try to produce the following graph:





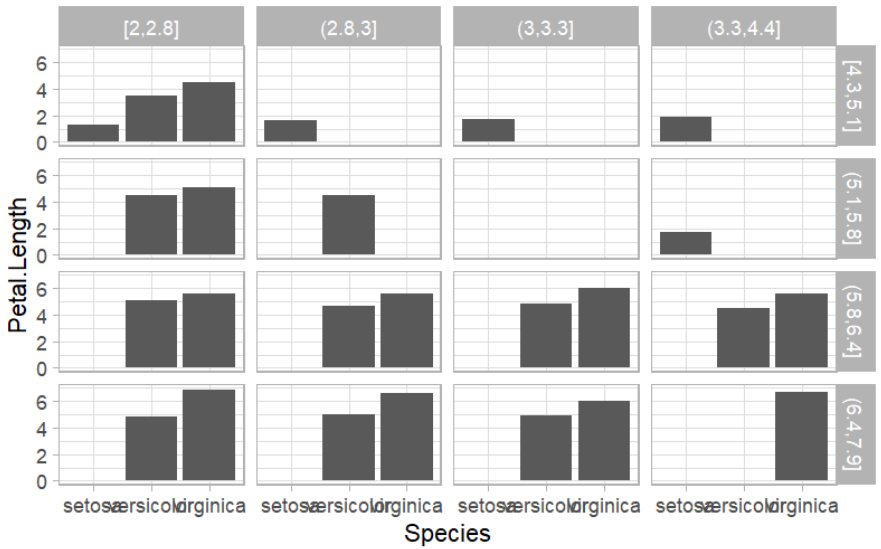
6) THEMES

Themes allows you to exercise fine **control** over **the non-data elements of your plot.** The theme system does not affect how the data is rendered by geoms, or how it is transformed by scales. Themes don't change the perceptual properties of the plot, but **they do help you make the plot aesthetically pleasing** or match an existing style guide. **Themes give you control over** things like **fonts, ticks, panel strips,** and **backgrounds.**



6) THEMES: PREDEFINED

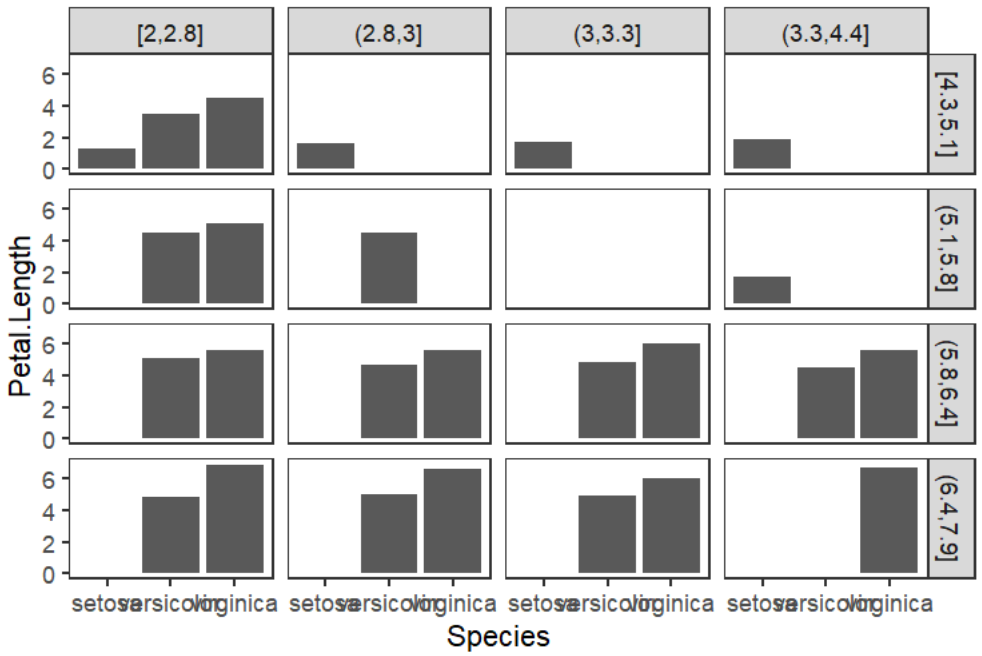
```
EX_3.4%>%
  ggplot(aes(Species,Petal.Length))+
  geom_col(position = "dodge")+
  facet_grid(Q_SL~Q_SW)+
  theme_light()
```

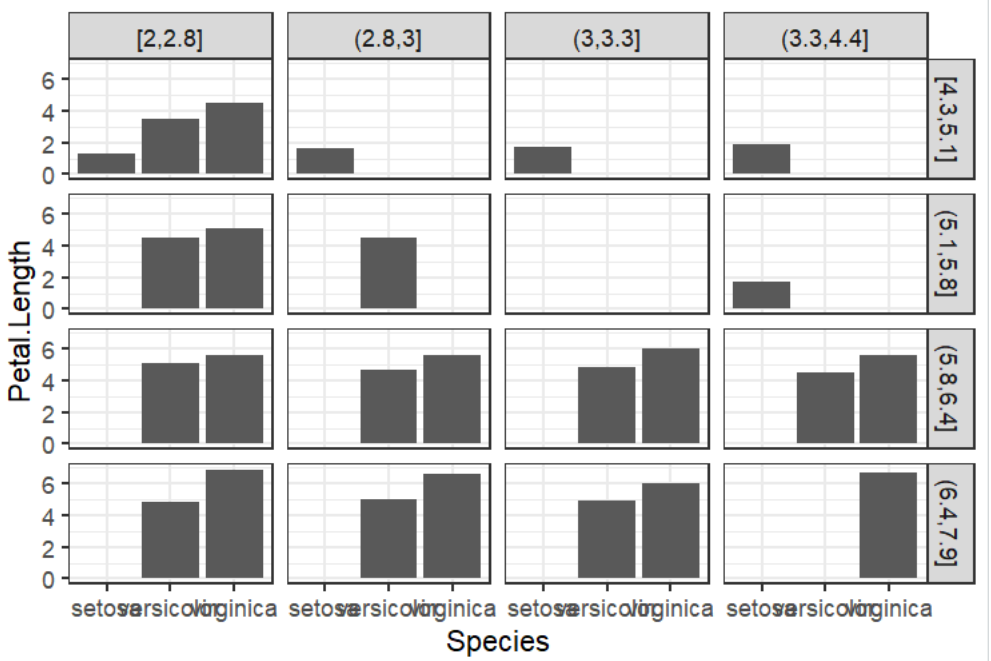


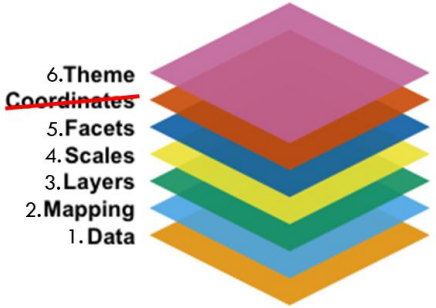
- `theme_bw()`: a variation on `theme_grey()` that uses a white background and thin grey grid lines.
- `theme_linedraw()`: A theme with only black lines of various widths on white backgrounds, reminiscent of a line drawing.
- `theme_light()`: similar to `theme_linedraw()` but with light grey lines and axes, to direct more attention towards the data.
- `theme_dark()`: the dark cousin of `theme_light()`, with similar line sizes but a dark background. Useful to make thin coloured lines pop out.
- `theme_minimal()`: A minimalistic theme with no background annotations.
- `theme_classic()`: A classic-looking theme, with x and y axis lines and no gridlines.
- `theme_void()`: A completely empty theme.

EXERCISE 2.5

Using the predefined themes, try to produce the following graphs:



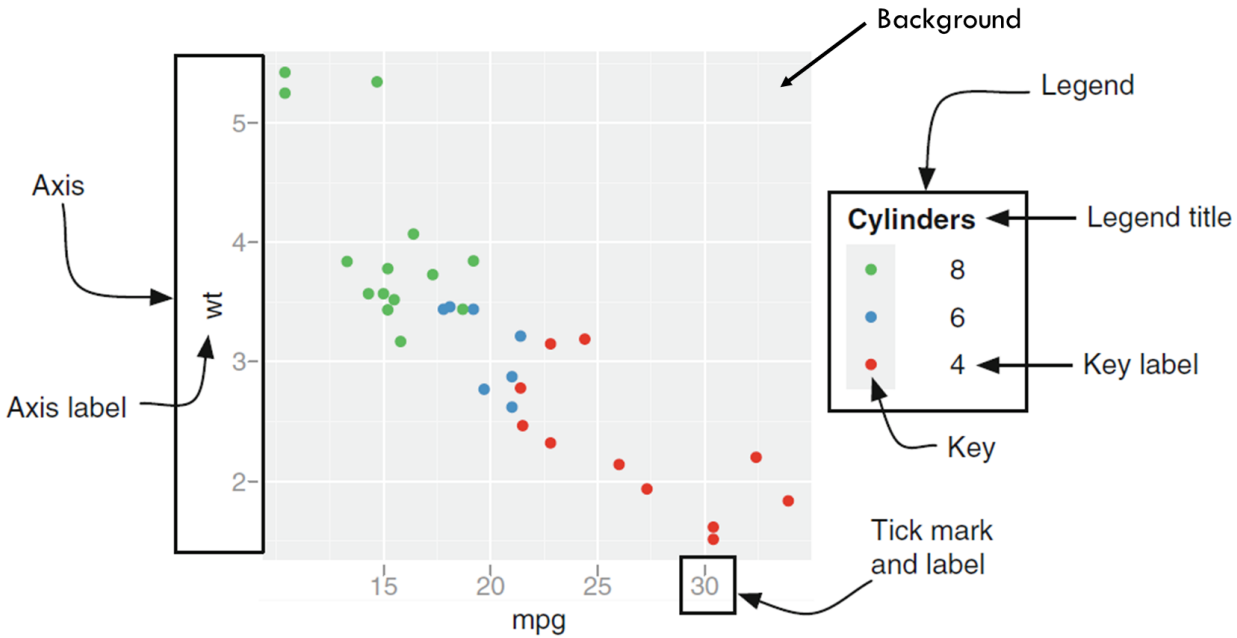


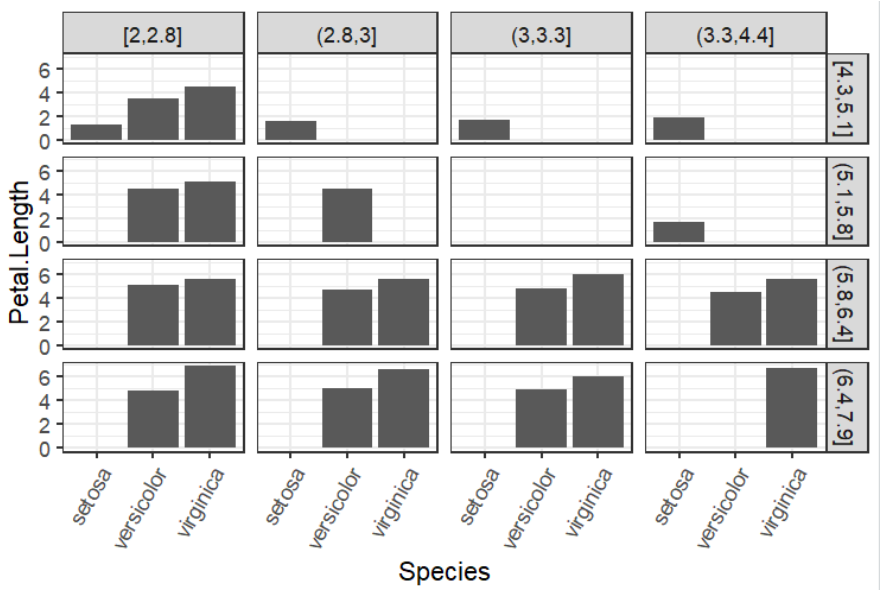


6) THEMES: MODIFYING SPECIFIC ASPECTS

You can use the ggplot2 function theme() to modify specific elements of the graph.

```
EX_3.4%>%
  ggplot(aes(Species,Petal.Length))+
  geom_col(position = "dodge")+
  facet_grid(Q_SL~Q_SW)+
  theme_bw()+
  theme(axis.text.x =
          element_text(angle = 60,hjust = 1))
```





EXERCISE 2.6

Play around with the parameters of the function “theme()” and produce the most horrible graph in the history of the course.

SAVING THE GRAPHS

ggplot2 provides a convenient shorthand to save graphs created with ggplot: **ggsave().**

Option 1:

```
EX_3.4%>%
  ggplot(aes(Species,Petal.Length))+
  geom_col(position = "dodge")+
  facet_grid(Q_SL~Q_SW)+
  theme_bw()+
  theme(axis.text.x =
          element_text(angle = 60,hjust = 1))

ggsave("NAME.png")
```

Option 2:

```
P1<-EX_3.4%>%
  ggplot(aes(Species,Petal.Length))+
  geom_col(position = "dodge")+
  facet_grid(Q_SL~Q_SW)+
  theme_bw()+
  theme(axis.text.x =
          element_text(angle = 60,hjust = 1))

ggsave(P1,"NAME.png")
```

EXERCISE 2.7

- In the “help” section, check the parameters of **ggsave.**
- Use **ggsave** to save your last graph. Play around with the parameters until you manage to save your graph in good quality format

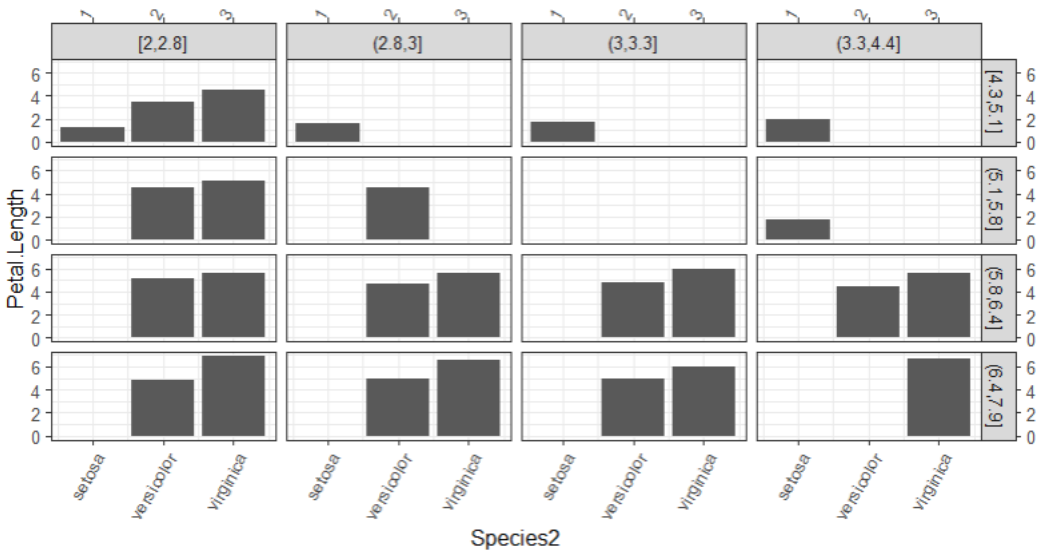
Hint: Remember to use the function paste0() to concatenate your folder path and the name of your graph.

EXERCISE 2.8: EXTRA FUNCTIONS

The following code:

```
EX_3.4%>%
  mutate(Species2=ifelse(Species=="setosa",1,
                         ifelse(Species=="versicolor",2,3)))%>%
  ggplot(aes(Species2,Petal.Length))+
  geom_col(position = "dodge")+
  facet_grid(Q_SL~Q_SW)+
  scale_y_continuous(
    sec.axis = sec_axis(~ . ))+
  scale_x_continuous(
    breaks = 1:3,
    labels = c("setosa","versicolor","virginica"),
    sec.axis = sec_axis(~ . ))+
  theme_bw()+
  theme(axis.text.x = element_text(angle = 60,hjust = 1))
```

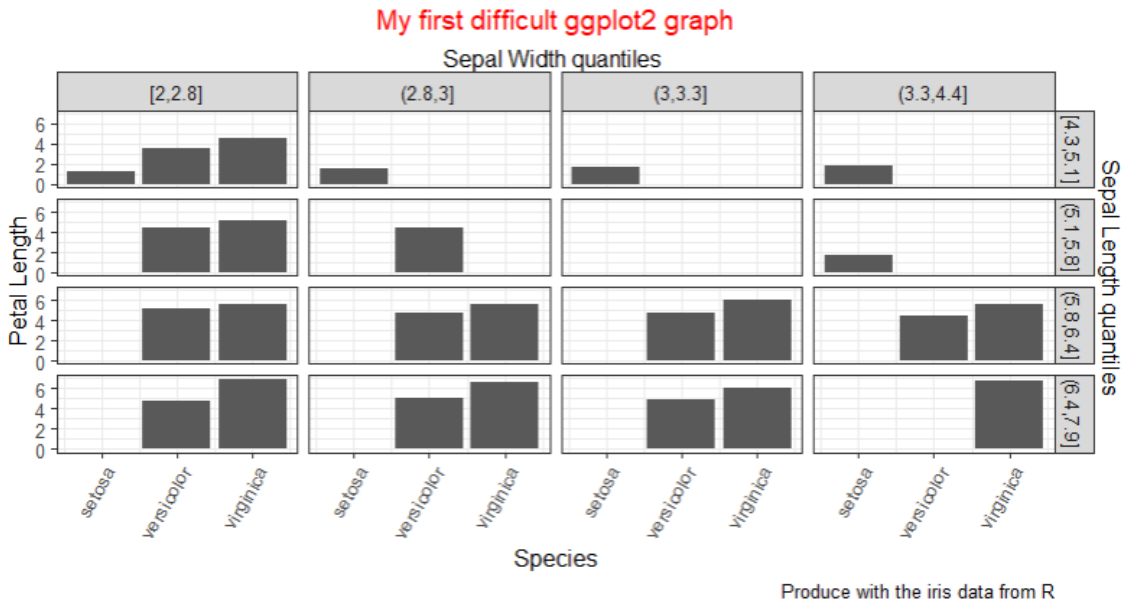
Produces:



Produce the following graph:

Hint: Check the parameters of the function: sec_axis() and labs().

Hint 2: Modify the parameters of the theme() function.



REFERENCES

- Wickham, Hadley, Mine Çetinkaya-Rundel, and Garrett Grolemund. *R for data science*. " O'Reilly Media, Inc.", 2023. Accessed May 7, 2024. https://r4ds.hadley.nz/.
- Wickham, Hadley. *Ggplot2*. Use R! Cham: Springer International Publishing, 2016. https://doi.org/10.1007/978-3-319-24277-4.
- Wilke, C. *Fundamentals of Data Visualization: A Primer on Making Informative and Compelling Figures*. First edition. Sebastopol, CA: O'Reilly Media, 2019. https://clauswilke.com/dataviz/

Join us!

https://forms.office.com/e/8Bgd2YsasJ

All about the lab:

https://societal-analytics.nl/

Contact us at:

analytics-lab.fsw@vu.nl.

https://sofiag1l.github.io/

THANKS!

Dr. Sofia Gil-Clavel